



**PROF. V. B. SHAH INSTITUTE OF MANAGEMENT,
R. V. PATEL COLLEGE OF COMMERCE (ENG. MED.),
V. L. SHAH COLLEGE OF COMMERCE (GUJ. MED.) &
SUTEX BANK COLLEGE OF COMPUTER APPLICATIONS & SCIENCE**

Managed by Jivan Jyot Trust, Amroli.

All Colleges are Affiliated to Veer Narmad South Gujarat University, Surat.

Self Financed B.B.A. - B.Com. (Eng. & Guj. Med.) - B.C.A & B.Sc. Data Science Degree Programme

Accredited by National Assessment and Accreditation Council with "B" Grade

Prin. Dr. Mukesh R. Goyani
M.Com., M.Ed., M.Phil., GSET, Ph.D.
Senate Member, VNSGU, Surat

At & Po. : Amroli, Surat,
Pin. 394107, Gujarat (India)

www.amrolicollege.ac.in

☎ (0261) 2494073, 2495643, 2496478
📠 9427113947, 6354626492
✉ principal@amrolicollege.ac.in
Principal_230@vnsgu.ac.in

PROJECT REPORT

ON

ADD YOUR PROJECT TITLE HERE

AS

A PARTIAL REQUIREMENT FOR THE DEGREE

OF

BACHELOR OF COMPUTER APPLICATION (BCA)

FOR SUBJECT

602 DATA ANALYSIS WITH PYTHON

Submitted by:

Student name (exam seat no.)

Student name (exam seat no.)

Guided by:

Asst. Prof. Twinkle S. Panchal

Sutex Bank College of Computer Applications and Science, Amroli

PROJECT Completion Certificate

Acknowledgement

INDEX

Sr. No.	Chapter title	Page no.
1	Project Definition	
2	Dataset Details	
3	Python tools and libraries used	
4	Understanding data	
5	Understanding Spread of Data	
6	Automating EDA using Python	
7	Handling missing data and outliers	
8	Univariate Analysis	
9	Bivariate Analysis	
10	Multivariate Analysis	
11	Data Visualization	
12	Python Code	
13	Conclusion	

Ch-1: Project Definition:

Understanding customer preferences and restaurant trends

Understanding customer preferences and restaurant trends is important for making informed business decisions in food industry. In this article, we will analyze Zomato's restaurant dataset using Python to find meaningful insights.

We aim to answer questions such as:

1. Identify popular restaurant categories.
2. Which restaurants are preferred by a larger number of individuals based on votes.
3. Find the restaurant with the highest number of votes.
4. Exploring the `online_order` column to see how many restaurants accept online orders.
5. Analyze distribution of customer rating.
6. What price range do couples prefer for dining?
7. Analyze customer ratings for online vs. offline Orders.
8. Analyze order mode (online/offline) preference by restaurant type.

Ch-2: Dataset Details

Dataset Name: Zomato restaurant data

Dataset Source:

<https://media.geeksforgeeks.org/wp-content/uploads/20250117023324808265/Zomato-data-.CSV>

Dataset Type: Structured CSV dataset

Dataset Domain: Restaurant Business Analytics

Dataset Description

The dataset contains information about:

- Restaurant Name
- Online order availability (yes/no)
- Table booking availability (yes/no)
- User rating for the restaurant
- Number of votes
- Approx meal cost for two people
- Restaurant type

This dataset is ideal for:

- EDA (Univariate, Bivariate, Multivariate)
- Handling missing values and outliers
- Data analysis using line chart, histogram, countplot, boxplot, heatmap etc.

Ch-3: Python tools and libraries used

Python 310

Python is a programming language widely used by Data Scientists. Python has in-built mathematical libraries and functions, making it easier to calculate mathematical problems and to perform data analysis.

Pandas

Pandas is a fundamental, open-source Python library for data manipulation and analysis, providing fast, flexible, and expressive data structures like DataFrames (2D tables) and Series (1D arrays) to handle tabular data efficiently, making it ideal for cleaning, transforming, exploring, and analyzing large datasets in data science, machine learning, and analytics. It simplifies tasks like reading/writing files (CSV, Excel), handling missing data, filtering, grouping, merging, and performing statistical analysis, integrating seamlessly with other tools like NumPy and Matplotlib.

NumPy

NumPy (Numerical Python) is a foundational Python library for data science, primarily used for its efficient multi-dimensional array object (the ndarray) and high-performance mathematical functions. It is a core component of the Python data science ecosystem, serving as the backbone for other major libraries like Pandas, Matplotlib, and scikit-learn.

Matplotlib

Matplotlib is a foundational, powerful Python library for creating a wide range of static, animated, and interactive visualizations in data science. It provides extensive customization options and serves as the backbone for many other visualization libraries like Seaborn.

Seaborn

Seaborn is a powerful Python library for statistical data visualization, built on Matplotlib, that creates attractive, informative plots with minimal code, seamlessly integrating with pandas DataFrames to reveal trends, relationships, and patterns in data, making it essential for Exploratory Data Analysis (EDA). It offers high-level functions for complex plots (like distributions, regressions, heatmaps, violin plots) with beautiful default styles, color palettes, and themes, simplifying the process of understanding data structures and distributions.

Scikit-learn

Scikit-learn (sklearn) is the most popular, open-source Python library for classical machine learning, providing simple and efficient tools for predictive data analysis. It is built on top of the scientific Python stack (NumPy, SciPy, Matplotlib) and is an essential tool for data scientists.

Ch-4: Understanding data

#Step 1: Importing necessary Python libraries.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

#Step 2: Creating the data frame.

```
dataframe = pd.read_csv("zomato-data.csv")
```

display all column names

```
print(dataframe.columns)
Index(['name', 'online_order', 'book_table', 'rate', 'votes',
       'approx_cost(for two people)', 'listed_in(type)',
       dtype='object')
```

setting pandas display option to display all columns

```
pd.set_option('display.max_columns', None)
```

display top few lines of data

```
print(dataframe.head(5))
```

	name	online_order	book_table	rate	votes
0	Jalsa	Yes	Yes	4.1/5	775.0
1	Spice Elephant	Yes	No	4.1/5	787.0
2	San Churro Cafe	Yes	No	3.8/5	918.0
3	Addhuri Udupi Bhojana	No	No	3.7/5	88.0
4	Grand Village	No	No	3.8/5	166.0

	approx_cost(for two people)	listed_in(type)
0	800.0	Buffet
1	800.0	Buffet
2	800.0	Buffet
3	300.0	Buffet
4	600.0	Buffet

display shape of data

```
print(dataframe.shape)
```

```
(148, 7)
```

Getting summary of the dataframe using df.info()

```
print("-----Dataset summary-----")
dataframe.info()
```


-----Dataset summary-----

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 148 entries, 0 to 147

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	name	148 non-null	object
1	online_order	148 non-null	object
2	book_table	148 non-null	object
3	rate	148 non-null	float64
4	votes	144 non-null	float64
5	approx_cost(for two people)	145 non-null	float64
6	listed_in(type)	148 non-null	object

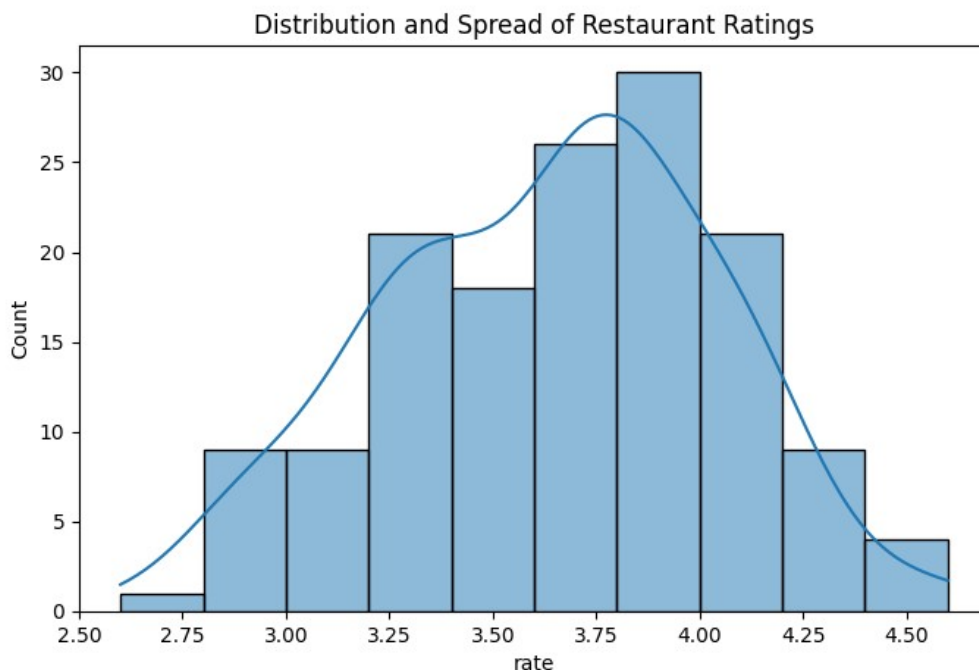
dtypes: float64(3), object(4)

memory usage: 8.2+ KB

Ch-5: Understanding spread of data

With our dataset we have used histogram with “kde=True” to understand the distribution and spread of data in column “rate” (user ratings).

```
# 1. Visualize the spread of 'rate' using a Histogram
plt.figure(figsize=(8, 5))
sns.histplot(dataframe['rate'], bins=10, kde=True)
plt.title("Distribution and Spread of Restaurant Ratings")
plt.show()
```



Further we have used boxplot to understand the data spread of “approximate cost for two people” by online order availability.

```
# 2. Compare the spread of 'approx_cost' across 'online_order'
categories using Box Plots
```

```
plt.figure(figsize=(8, 5))
sns.boxplot(x='online_order', y='approx_cost(for two people)',
data=dataframe)
plt.title("Cost Spread by Online Order Availability")
plt.show()
```



Ch-6: Automating EDA using python

Automating Exploratory Data Analysis (EDA) in Python is achieved using specialized libraries that generate comprehensive, interactive reports with just a few lines of code.

In Python's pandas library, `df.info()` and `df.describe()` are fundamental methods for performing initial exploratory data analysis (EDA).

df.info(): Provides a concise structural summary of the DataFrame. It provides following information about dataset:

- Data Types (Dtypes),
- Non-Null Counts,
- Column Names,
- Memory Usage,
- Total Entries/Rows

df.describe(): Offers a statistical summary of the numerical data.

- Count: The number of non-null observations in each column.
- Mean: The average value.
- Std: The standard deviation (spread of data).
- Min: The minimum value.
- 25%, 50%, 75%: The first quartile, median (50th percentile), and third quartile.
- Max: The maximum value.

These functions help data analysts quickly understand the dataset's composition and spot potential issues like missing values or outliers without manually sifting through rows.

```
# Step 4: Getting summary of the dataframe using df.info()
print("-----Dataset summary-----")
dataframe.info()
```

```
-----Dataset summary-----
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 148 entries, 0 to 147
Data columns (total 7 columns):
 #   Column                                  Non-Null Count  Dtype
---  -
 0   name                                   148 non-null    object
 1   online_order                           148 non-null    object
 2   book_table                             148 non-null    object
 3   rate                                   148 non-null    float64
 4   votes                                 144 non-null    float64
 5   approx_cost(for two people)           145 non-null    float64
 6   listed_in(type)                       148 non-null    object
dtypes: float64(3), object(4)
memory usage: 8.2+ KB
```

```
# Step 4.1: Getting description of the dataframe using df.describe()
print("-----Dataset description-----")
print(dataframe.describe())
```

```
-----Dataset description-----
              rate      votes  approx_cost(for two people)
count  148.000000   144.000000                145.000000
mean     3.633108   265.729167                415.517241
std     0.402271   662.490314                221.697777
min     2.600000     0.000000                100.000000
25%     3.300000     6.000000                200.000000
50%     3.700000    39.000000                400.000000
75%     3.900000   205.500000                600.000000
max     4.600000  4884.000000                950.000000
```

Ch-7: Handling missing data and outliers (Data cleaning and preparation)

Data Cleaning

Before moving further we need to clean and process the data. In this dataset the “rate” column contains data in string format having values like “<rating> / 5”. Here “ /5” is unwanted string (noise in data), which needs to be removed. And then the rating has to be transformed into float data type so that the rating data can be used for analysis.

Convert the rate column to a float by removing denominator characters.

```
dataframe['rate']=dataframe['rate'].apply(handleRate):
```

Applies the handleRate function to clean and convert each rating value in the 'rate' column.

```
def handleRate(value):  
    value=str(value).split('/')  
    value=value[0];  
    return float(value)
```

```
dataframe['rate']=dataframe['rate'].apply(handleRate)
```

```
print("-----After cleaning rate column, printing top records")  
print(dataframe[['name','rate']].head())
```

```
-----After cleaning rate column, printing top few records  
   name      rate  
0   Jalsa    4.1  
1  Spice Elephant  4.1  
2  San Churro Cafe  3.8  
3  Addhuri Udupi Bhojana  3.7  
4   Grand Village    3.8
```

As far as outliers are concerned, the outlier values are not incorrect data in this dataset and hence are meaningful for this analysis.

Checking for missing values

Step 5: Checking for missing or null values to identify any data gaps (EDA)

```
print("-----Checking for missing or null values")
print(dataframe.isnull().sum())
```

```
-----Checking for missing or null values
name                                0
online_order                       0
book_table                         0
rate                              0
votes                             4
approx_cost(for two people)        3
listed_in(type)                   0
dtype: int64
```

Handling missing values

There are a few missing values found in “votes” (number of votes for restaurant) and “approx_cost (for two people)” columns. We have filled up missing values using mean values.

```
df_cleaned = dataframe.fillna(dataframe.mean(numeric_only=True))
print("displaying null value count after handling missing data with
mean value")
print(df_cleaned.isnull().sum())
dataframe = df_cleaned
```

```
displaying null value count after handling missing data with mean value
name                                0
online_order                       0
book_table                         0
rate                              0
votes                             0
approx_cost(for two people)        0
listed_in(type)                   0
dtype: int64
```

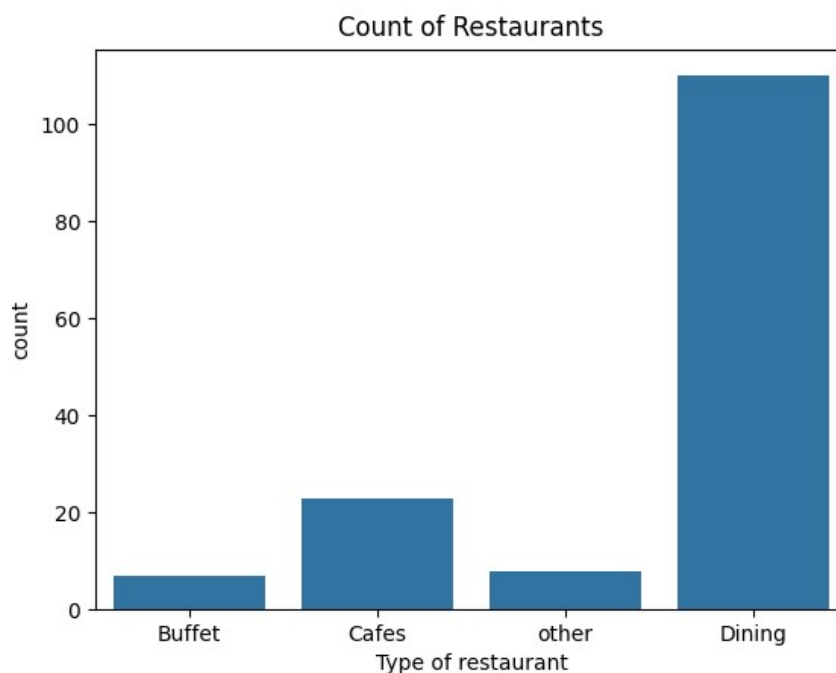
Ch-8 Univariate Analysis

Here we are exploring restaurant types as a part of univariate analysis under Exploratory Data Analysis. This will answer our question 1: Identify popular restaurant categories.

We are using `countplot()` for this purpose. `countplot()` using Seaborn is commonly used in EDA to visualize categorical data. A countplot shows the count (frequency) of each unique category in a categorical variable using bars.

Step 7: Exploring Restaurant Types (EDA - Univariate Analysis)

```
sns.countplot(x=dataframe['listed_in(type)'])
plt.title("Count of Restaurants")
plt.xlabel("Type of restaurant")
plt.show()
```



Conclusion: The majority of the restaurants fall into the dining category.

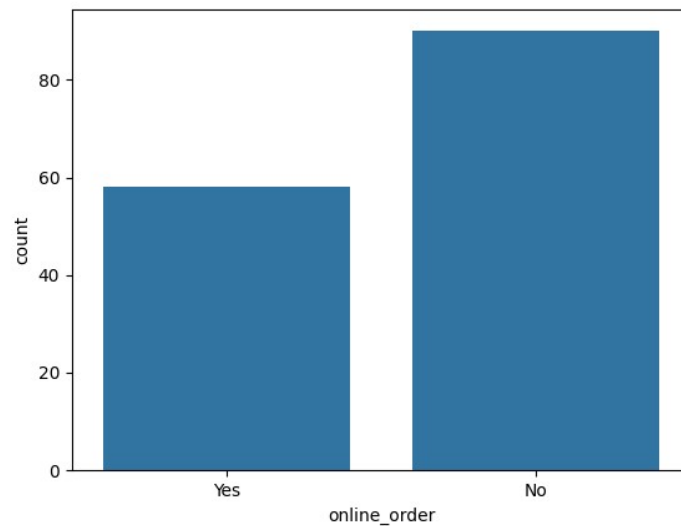
Step 9: Identify the Most Voted Restaurant

```
max_votes = dataframe['votes'].max()
restaurant_with_max_votes = dataframe.loc[dataframe['votes'] ==
max_votes, 'name']
print('Restaurant(s) with the maximum votes:')
print(restaurant_with_max_votes)
```

Conclusion: Empire State restaurant is most voted restaurant

Step 10: Online Order Availability

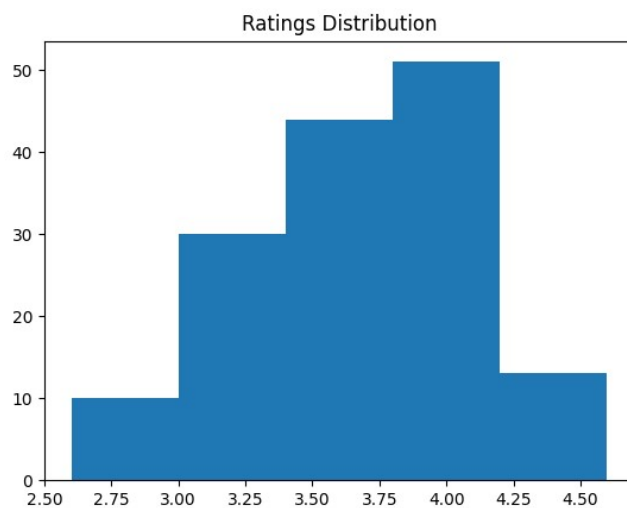
```
sns.countplot(x=dataframe['online_order'])  
plt.show()
```



Conclusion: majority of the restaurants do not accept online orders.

Step 11: Analyze customer ratings Ratings

```
plt.hist(dataframe['rate'],bins=5)  
plt.title('Ratings Distribution')  
plt.show()
```

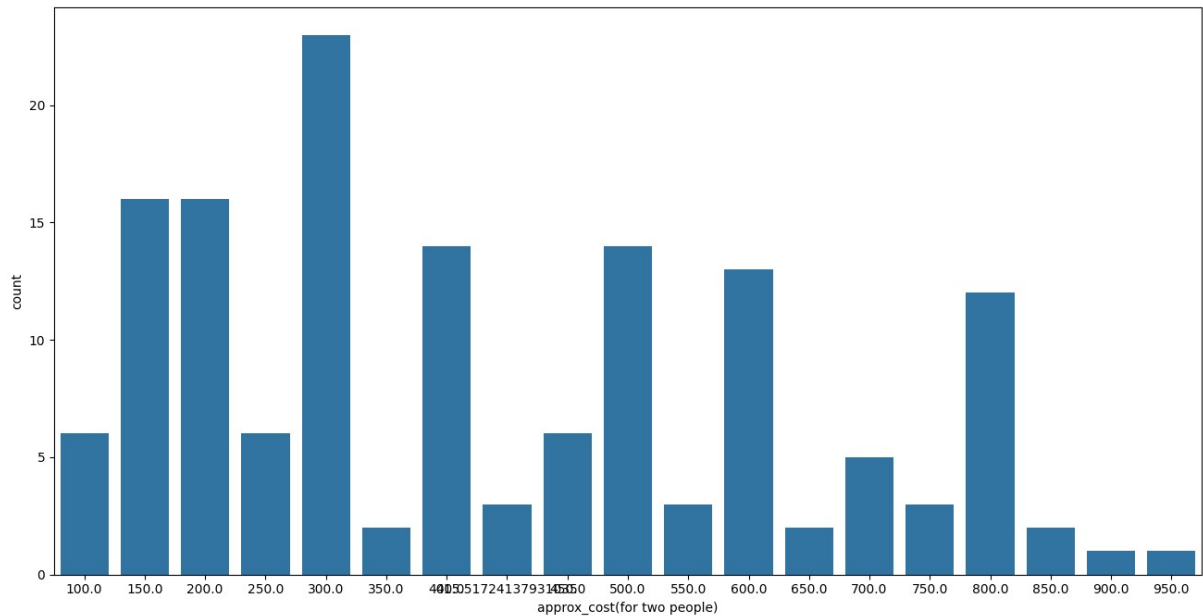


Conclusion: The majority of restaurants received ratings ranging from 3.5 to 4.

Step 12: Approximate Cost of dining for Couples

Analysing approx_cost(for two people) column to find the preferred price range

```
couple_data=dataframe['approx_cost(for two people)']  
sns.countplot(x=couple_data)  
plt.show()
```



Conclusion: The majority of couples prefer restaurants with an approximate cost of 300 rupees.

Ch-9 Bivariate Analysis

Here we are exploring user votes by restaurant type as a part of bivariate analysis under Exploratory Data Analysis. This will answer our question 2: Which restaurants are preferred by a larger number of individuals based on votes.

First we are using `grouby()` function to find restaurant category wise total user votes data. And then we are using line chart to represent this data visually.

Step 8: Exploring Votes by Restaurant Type (EDA - Bivariate Analysis)

```
grouped_data = dataframe.groupby('listed_in(type)')['votes'].sum()
print(grouped_data)
result = pd.DataFrame({'votes': grouped_data})
print(result)
```

	votes
listed_in(type)	
Buffet	3028.000000
Cafes	6415.458333
Dining	20495.729167
other	9388.729167

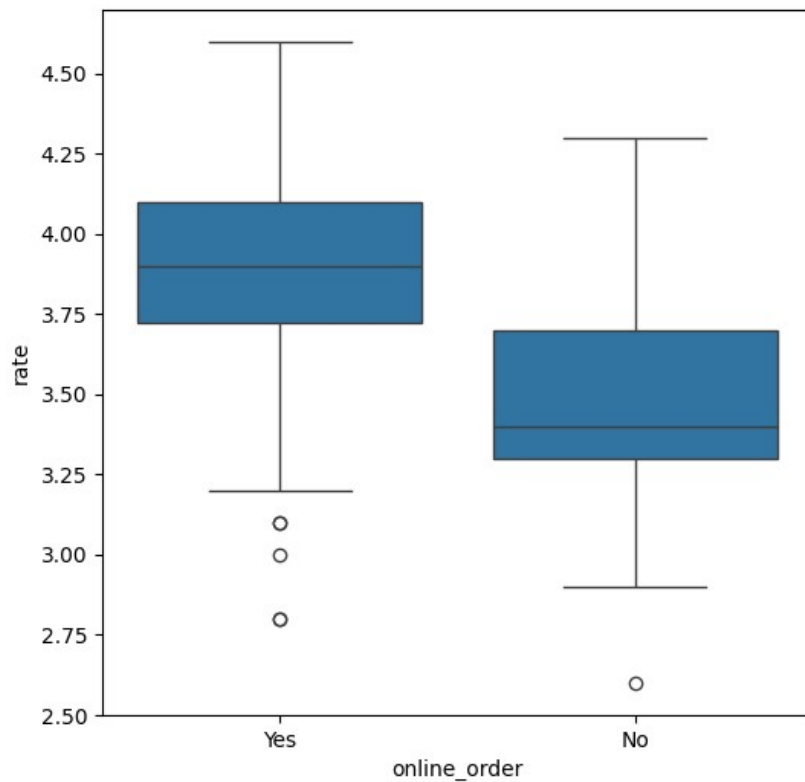
```
plt.plot(result, c='green', marker='o')
plt.xlabel('Type of restaurant')
plt.ylabel('Votes')
plt.show()
```



Conclusion: Dining restaurants are preferred by a larger number of individuals.

Step 13: Ratings Comparison - Online vs Offline Orders

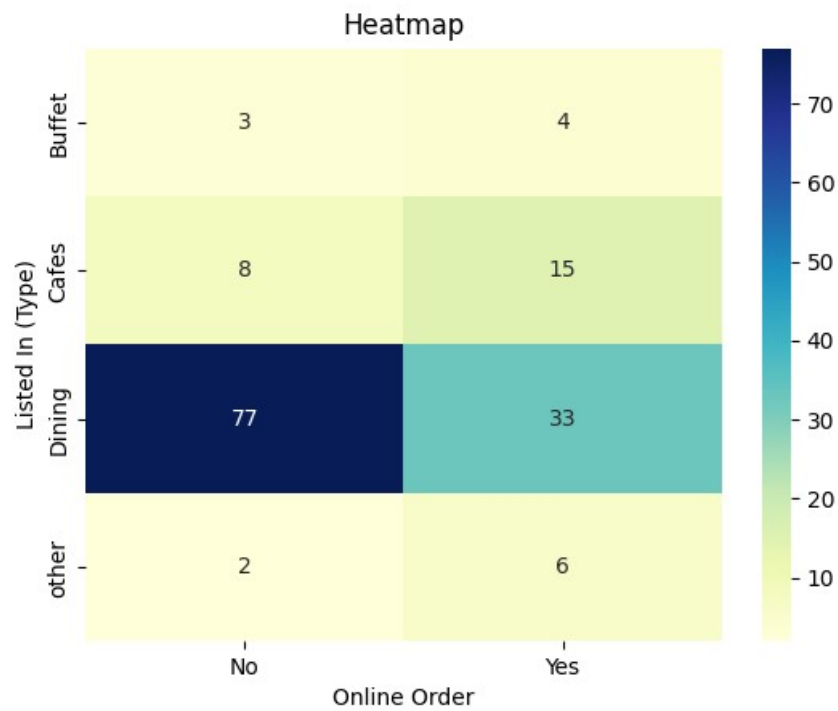
```
plt.figure(figsize = (6,6))
sns.boxplot(x = 'online_order', y = 'rate', data = dataframe)
plt.show()
```



Conclusion: Offline orders received lower ratings in comparison to online orders which obtained excellent ratings.

Step 14: Order Mode Preferences by Restaurant Type

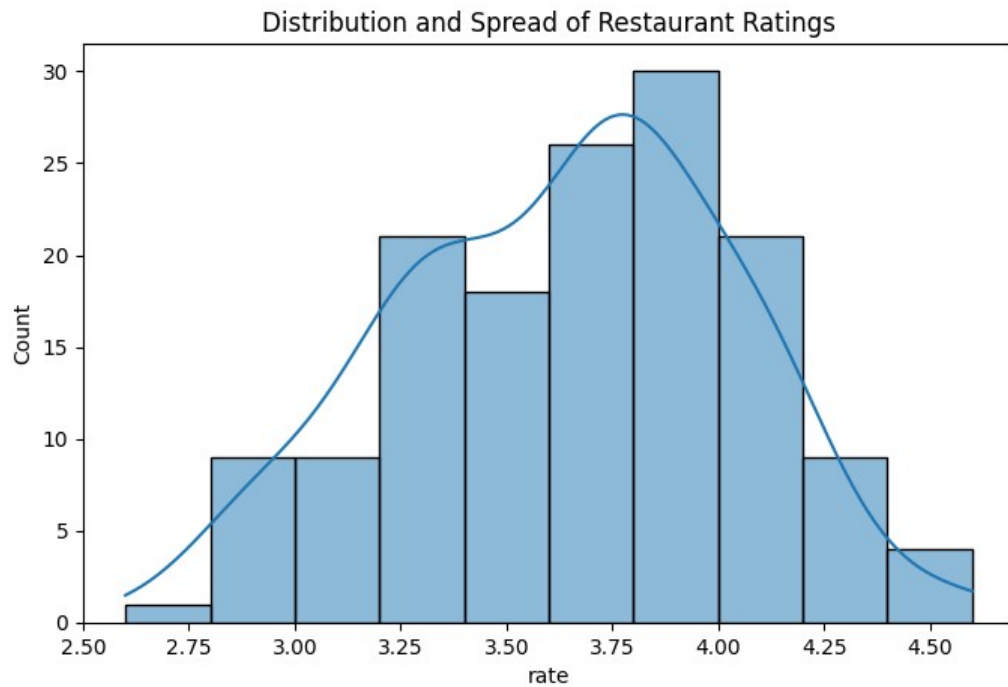
```
pivot_table = dataframe.pivot_table(index='listed_in(type)',
columns='online_order', aggfunc='size', fill_value=0)
sns.heatmap(pivot_table, annot=True, cmap='YlGnBu', fmt='d')
plt.title('Heatmap')
plt.xlabel('Online Order')
plt.ylabel('Listed In (Type)')
plt.show()
```



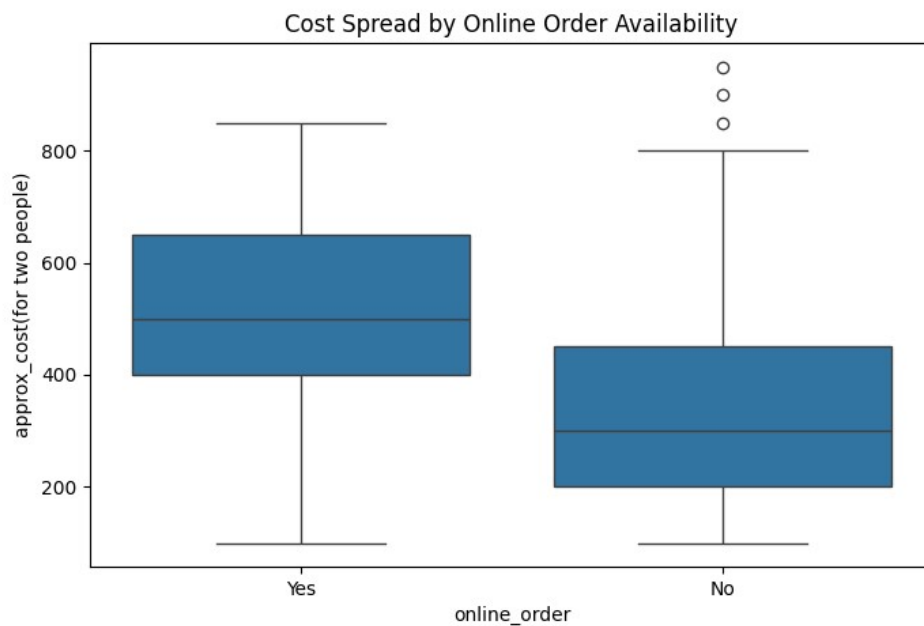
Conclusion: With this we can say that dining restaurants primarily receive offline orders whereas cafes primarily receive online orders. This suggests that clients prefer to place orders in person at dining restaurants but prefer online ordering at cafes.

Ch-11 Data Visualization

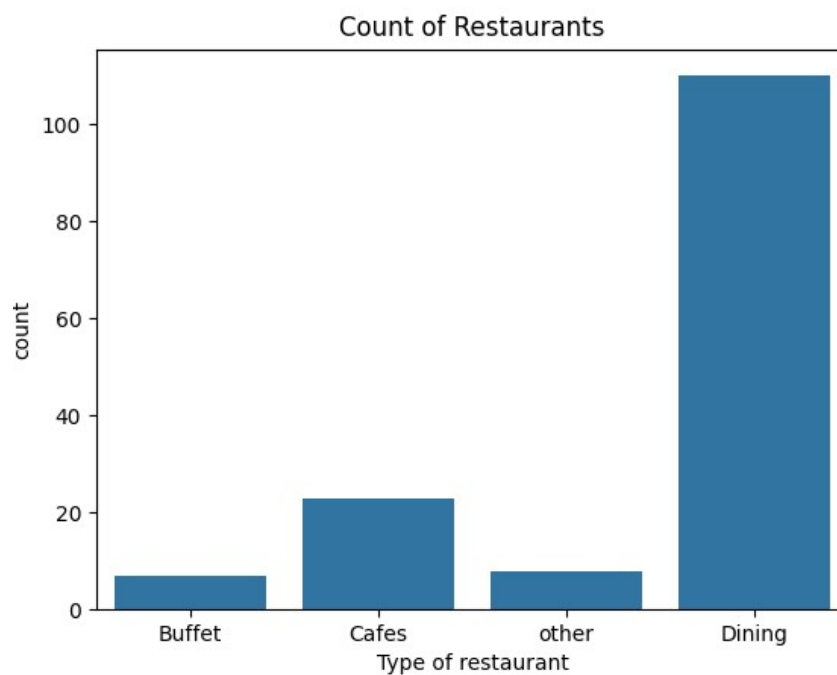
1. Visualize the spread of 'rate' using a Histogram



2. Compare the spread of 'approx_cost' across 'online_order' categories using Box Plots

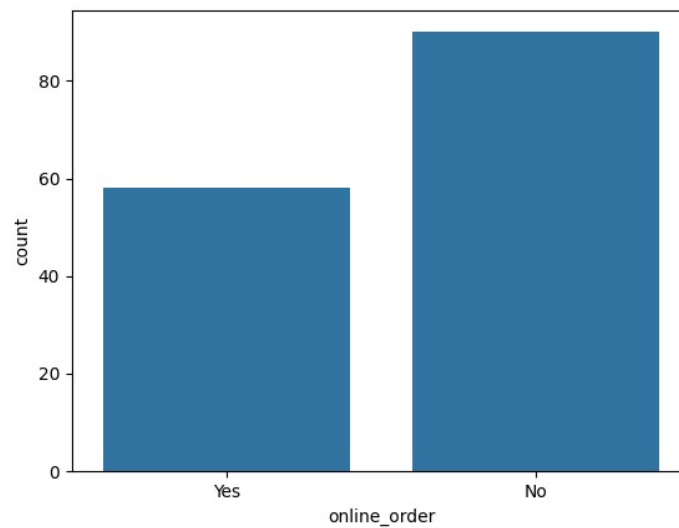


3: Exploring Restaurant Types (EDA - Univariate Analysis)



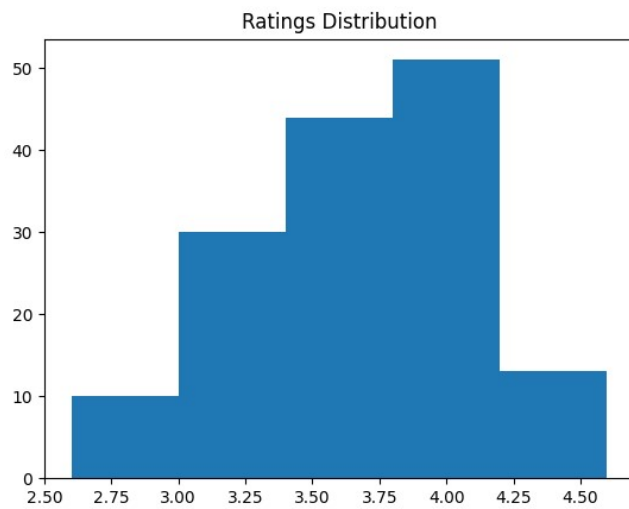
Conclusion: The majority of the restaurants fall into the dining category.

4: Online Order Availability



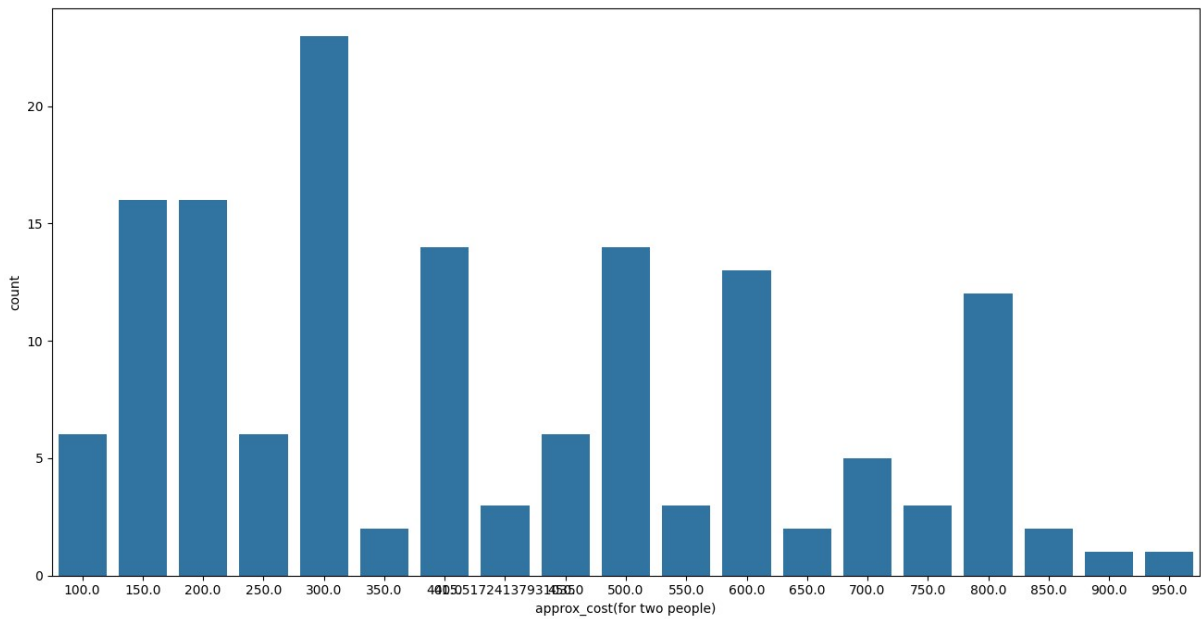
Conclusion: majority of the restaurants do not accept online orders.

5: Analyze customer ratings Ratings



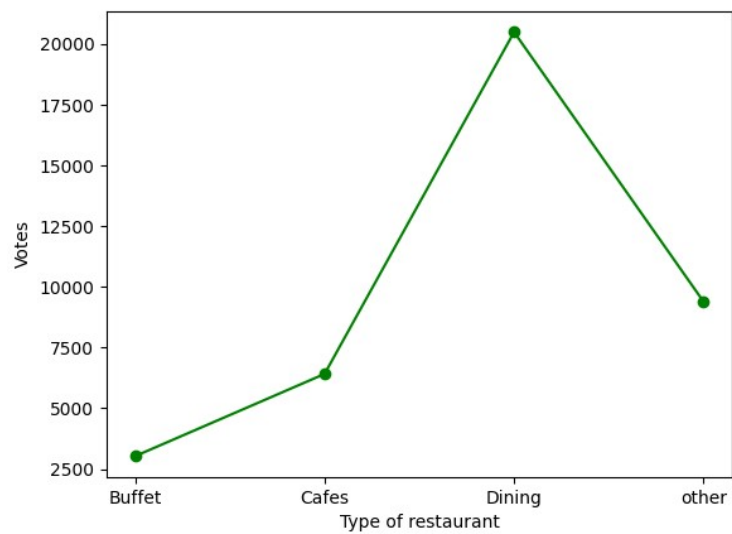
Conclusion: The majority of restaurants received ratings ranging from 3.5 to 4.

6: Approximate Cost of dining for Couples



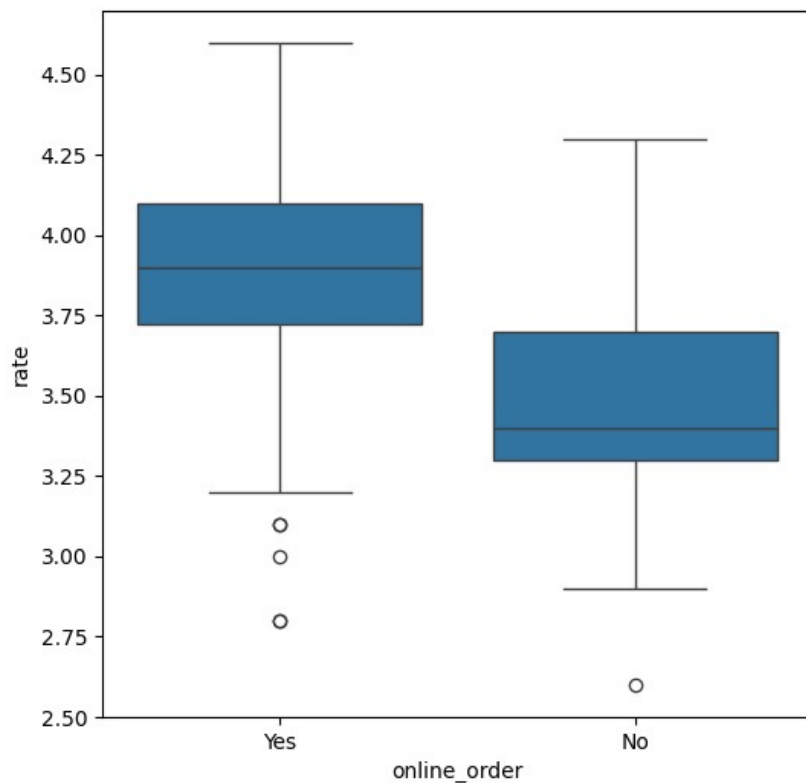
Conclusion: The majority of couples prefer restaurants with an approximate cost of 300 rupees.

7: Exploring Votes by Restaurant Type



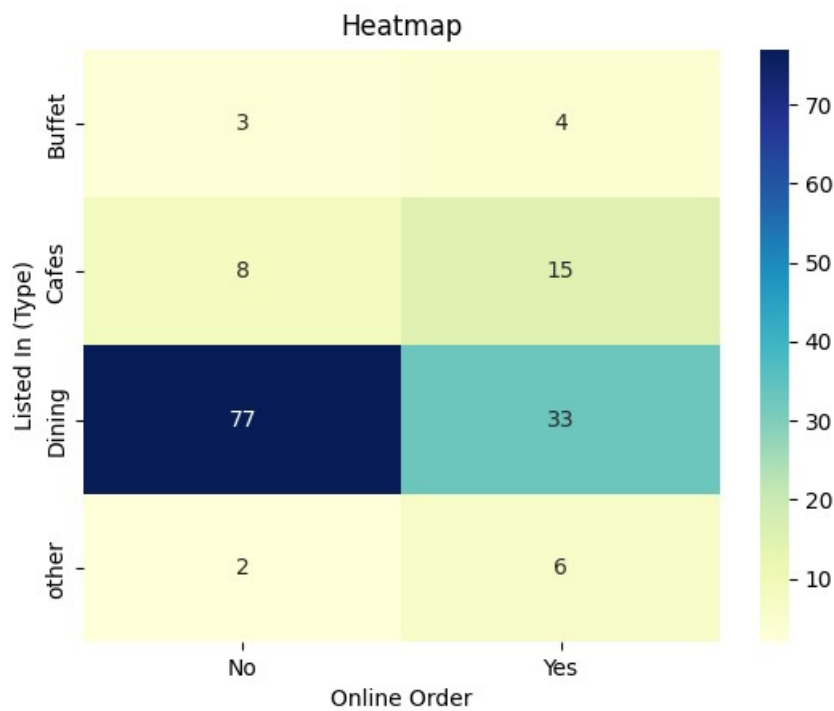
Conclusion: Dining restaurants are preferred by a larger number of individuals.

8: Ratings Comparison - Online vs Offline Orders



Conclusion: Offline orders received lower ratings in comparison to online orders which obtained excellent ratings.

9: Order Mode Preferences by Restaurant Type



Conclusion: With this we can say that dining restaurants primarily receive offline orders whereas cafes primarily receive online orders. This suggests that clients prefer to place orders in person at dining restaurants but prefer online ordering at cafes.

Ch-12 Python Code

```
#Step 1: Importing necessary Python libraries.
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
#Step 2: Creating the data frame.
dataframe = pd.read_csv("zomato-data.csv")
# display all column names
print(dataframe.columns)
# setting pandas display option to display all columns
pd.set_option('display.max_columns', None)
# display top few lines of data
print(dataframe.head(5))
# display shape of data
print(dataframe.shape)
#Step 3: Data Cleaning and Preparation
def handleRate(value):
    value=str(value).split('/')
    value=value[0];
    return float(value)
dataframe['rate']=dataframe['rate'].apply(handleRate)
print("-----After cleaning rate column, printing top few
records")
print(dataframe[['name','rate']].head())
# Step 4: Getting summary of the dataframe using df.info()
print("-----Dataset summary-----")
dataframe.info()
# Step 4.1: Getting description of the dataframe using df.describe()
print("-----Dataset description-----")
print(dataframe.describe())
# Step 5: Checking for missing or null values to identify any data
gaps (EDA)
```

```

print("-----Checking for missing or null values")
print(dataframe.isnull().sum())
# Step 6: Handling missing data: filling gaps with mean value (Data
Preparation)
df_cleaned = dataframe.fillna(dataframe.mean(numeric_only=True))
print("displaying null value count after handling missing data with
mean value")
print(df_cleaned.isnull().sum())
dataframe = df_cleaned
# step 6.1: understanding spread of data
# 1. Visualize the spread of 'rate' using a Histogram
plt.figure(figsize=(8, 5))
sns.histplot(dataframe['rate'], bins=10, kde=True)
plt.title("Distribution and Spread of Restaurant Ratings")
plt.show()
# 2. Compare the spread of 'approx_cc' across 'online_or' categories
using Box Plots
plt.figure(figsize=(8, 5))
sns.boxplot(x='online_order', y='approx_cost(for two people)',
data=dataframe)
plt.title("Cost Spread by Online Order Availability")
plt.show()
# Step 7: Exploring Restaurant Types (EDA - Univariate Analysis)
sns.countplot(x=dataframe['listed_in(type)'])
plt.title("Count of Restaurants")
plt.xlabel("Type of restaurant")
plt.show()
# Step 8: Exploring Votes by Restaurant Type (EDA - Bivariate
Analysis)
grouped_data = dataframe.groupby('listed_in(type)')['votes'].sum()
print(grouped_data)
result = pd.DataFrame({'votes': grouped_data})
print(result)
plt.plot(result, c='green', marker='o')

```

```

plt.xlabel('Type of restaurant')
plt.ylabel('Votes')
plt.show()
# Step 9: Identify the Most Voted Restaurant
max_votes = dataframe['votes'].max()
restaurant_with_max_votes = dataframe.loc[dataframe['votes'] ==
max_votes, 'name']
print('Restaurant(s) with the maximum votes:')
print(restaurant_with_max_votes)
# Step 10: Online Order Availability
sns.countplot(x=dataframe['online_order'])
plt.show()
# Step 11: Analyze customer Ratings
plt.hist(dataframe['rate'],bins=5)
plt.title('Ratings Distribution')
plt.show()
# Step 12: Approximate Cost for Couples
# Analysing approx_cost(for two people) column to find the preferred
price range
couple_data=dataframe['approx_cost(for two people)']
sns.countplot(x=couple_data)
plt.show()
# Step 13: Ratings Comparison - Online vs Offline Orders
plt.figure(figsize = (6,6))
sns.boxplot(x = 'online_order', y = 'rate', data = dataframe)
plt.show()
# Step 14: Order Mode Preferences by Restaurant Type
pivot_table = dataframe.pivot_table(index='listed_in(type)',
columns='online_order', aggfunc='size', fill_value=0)
sns.heatmap(pivot_table, annot=True, cmap='YlGnBu', fmt='d')
plt.title('Heatmap')
plt.xlabel('Online Order')
plt.ylabel('Listed In (Type)')
plt.show()

```

Ch-13 Conclusion

By applying different data analysis techniques, and based on the statistical and visual results, we have reached on following conclusions.

1. The majority of the restaurants fall into the dining category.
2. Dining restaurants are preferred by a larger number of individuals.
3. Empire State restaurant is most voted restaurant.
4. Majority of the restaurants do not accept online orders.
5. The majority of restaurants received ratings ranging from 3.5 to 4.
6. The majority of couples prefer restaurants with an approximate cost of 300 rupees.
7. Offline orders received lower ratings in comparison to online orders which obtained excellent ratings.
8. With this we can say that dining restaurants primarily receive offline orders whereas cafes primarily receive online orders. This suggests that clients prefer to place orders in person at restaurants but prefer online ordering at cafes.

Ch-14 References

1. Python official documentation: <https://docs.python.org/3/>
2. Pandas official documentation <https://pandas.pydata.org/docs/>
3. Numpy official documentation: <https://numpy.org/doc/>
4. Seaborn official documentation: <https://seaborn.pydata.org/>
5. Matplotlib official documentation:
<https://matplotlib.org/stable/index.html>
6. Scikit learn official documentation: <https://scikit-learn.org/stable/>
7. Kaggle AI/ML community: <https://www.kaggle.com/>
8. UCI Machine learning repository: <https://archive.ics.uci.edu/>
9. Machine learning tutorial: <https://www.geeksforgeeks.org/machine-learning/machine-learning/>